Code-Layout, Readability and Reusability

Date	14 July, 2026
Team ID	
Project Name	Credit Card Approval Screening (Flask App - app.py)
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the code quality of the Credit Card Approval Screening Flask application (**app.py**) in terms of structure, readability, maintainability, and reusability. The application is built with Python and Flask, and serves a trained scikit-learn model for credit-risk scoring, with routes for prediction, a dashboard, a what-if sandbox, and an analyst notification queue. The assessment below reflects the actual code as written, noting both its strengths and areas that would benefit from further refactoring.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	**Consistent Indentation**	Uniform 4-space indentation and PEP 8-style spacing maintained throughout app.py.	Yes	Clean, easy to scan and debug.
2	**Proper File Structure**	Model artifacts loaded from a models/ folder, templates rendered via render_template, and applicant data read from a data/ folder, matching standard Flask conventions.	Yes	Follows Flask project layout conventions.
3	**Meaningful Variable Names**	Variables such as feature_columns, shap_background, proba_high_risk, and BORDERLINE_LOW/HIGH clearly convey their purpose.	Yes	Easy to follow the scoring logic.
4	**Function / Method Names**	Functions like build_feature_row(), explain(), and score_application() are descriptively named; however, there is no separate load_model() or preprocess_data() function - model loading and preprocessing are inlined at module level.	Partial	Names are clear, but loading/preprocessing aren't encapsulated in their own functions.
5	**Code Comments**	A module-level docstring documents all routes, and inline comments explain the SHAP fallback, borderline band, and in-memory notification store. Individual route handlers (dashboard, sandbox, api_metrics) have little to no inline explanation.	Partial	Good high-level docs; per-route comments are sparse.
6	**Modular Design**	Model loading, feature engineering, explanation, scoring, and all Flask routes live in a single app.py file. Data preprocessing/training and the web layer are not split into separate modules or packages.	Partial	Works as a demo, but a production version should split model/, utils.py, and routes into separate modules.

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
7	**No Redundant Code**	score_application(), build_feature_row(), and explain() are reused by the /predict route and could be reused by other routes, avoiding duplicated scoring logic.	Yes	Central scoring pipeline avoids duplication.
8	**Error Handling**	try/except is used only around optional SHAP import and background-data loading. The /predict route and build_feature_row() do not validate or catch malformed payloads (e.g. float(payload.get(col, 0)) will raise on non-numeric input), and the app runs with debug=True.	Partial	Import-time errors are handled; request-level input validation is missing.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Used	Reusability Level (High / Medium / Low)
1	**Flask Web Application (routes: /, /predict, /dashboard, /sandbox, /notifications)**	Flask, Jinja templates	User interface / API layer	High
2	**build_feature_row()**	pandas	Converting form/JSON input into a model-ready feature row	High
3	**score_application()**	scikit-learn, joblib	Shared scoring pipeline used by /predict	High
4	**explain()**	SHAP (optional) with global-importance fallback	Per-prediction / fallback explanations	Medium
5	**Trained ML artifacts (best_model.joblib, scaler.joblib)**	Random Forest / Logistic Regression (via scikit-learn)	Prediction system	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clean, consistently formatted single-file Flask app; artifacts and templates are organized, though everything (model loading, feature engineering, routes) sits in one app.py.
Readability	4	Descriptive variable/function names and a clear module docstring make the flow easy to follow; individual route handlers could use a bit more inline commentary.
Reusability	4	Core scoring logic (build_feature_row, score_application, explain) is factored out and reused by /predict, but isn't yet packaged as an importable module for reuse outside this app.

Aspect	Rating (1-5)	Comments
Documentation / Comments	4	Good top-level documentation of routes and rationale (SHAP fallback, borderline band); comments thin out inside individual route handlers but the overall intent stays clear.
Error Handling	4	Import/loading of SHAP is guarded with try/except and the app fails gracefully to a fallback explainer; tightening request-level input validation would make this fully robust.
Overall Score	4 / 5	Solid, readable, well-organized code with sensible shared functions; moving preprocessing/model logic into separate modules and adding stricter input validation would push it toward a 5/5 production standard.